

Week 2 Reading: Steady-State MLPs

1. One reactor, three steady-state outputs

For consecutive first-order reactions $A \rightarrow B \rightarrow C$, setting each derivative to zero lets us solve in the order A, B, then C:

$$\begin{aligned}C_A &= \frac{C_{Af}}{1 + k_1\tau} \\C_B &= \frac{k_1\tau C_{Af}}{(1 + k_1\tau)(1 + k_2\tau)} \\C_C &= k_2\tau C_B\end{aligned}$$

At 330 K, the first and second reaction rates are 0.2/min and 0.1/min. With 5 min residence time and 1.2 mol/L feed, the three concentrations are **[0.6, 0.4, 0.2] mol/L**. Their sum is 1.2. These are the labels for one steady-state training row.

The input is $[T, \tau, C_{Af}]$ and the output is $[C_A, C_B, C_C]$. Each row is one steady operating condition. State transitions and future input histories are introduced in Week 3.

2. Count parameters before training

A dense layer has one weight for each input–output pair and one bias per output. The steady MLP is $3 \rightarrow 32 \rightarrow 32 \rightarrow 3$:

Layer	Count
First hidden layer	$3 \times 32 + 32 = 128$
Second hidden layer	$32 \times 32 + 32 = 1056$
Output layer	$32 \times 3 + 3 = 99$
Total	1283

$$h_1 = \text{ReLU}(W_1 \tilde{x} + b_1)$$

$$h_2 = \text{ReLU}(W_2 h_1 + b_2)$$

$$\hat{z} = W_3 h_2 + b_3$$

ReLU applies $\max(0, \text{value})$ to each hidden pre-activation. A linear output can return negative concentrations; nonnegativity and concentration-sum consistency must therefore be evaluated.

3. Scaling changes what the loss emphasizes

Use the declared bounds for input scaling. For output standardization, compute the mean and standard deviation from training data only; restore physical units before reporting concentration errors.

$$\tilde{x}_j = 2 \frac{x_j - l_j}{u_j - l_j} - 1$$
$$z_j = \frac{y_j - \mu_j}{s_j}, \quad \hat{y}_j = \mu_j + s_j \hat{z}_j$$

$$L = \frac{1}{3N} \sum_{n=1}^N \sum_{j=1}^3 \left(\hat{z}_{n,j} - z_{n,j} \right)^2$$

A raw squared error is weighted by the inverse squared training standard deviation. With standard deviations approximately 0.283 for A and 0.110 for B, the same raw squared error receives about **6.62 times** the weight for B compared with A.

This explains a weighting choice, not a guarantee of better predictions. Compare each component's test RMSE and the concentration-sum residual. See the [official MSE definition](#).

4. One backward pass, one parameter update

Use a $3 \rightarrow 2 \rightarrow 3$ network with $W_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$, $W_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$, and zero biases. Input $[1, 0, -1]$ gives hidden pre-activation $[1, -1]$, ReLU output $[1, 0]$, and prediction $[1, 0, 1]$. Against target $[0, 1, 0]$, the error is $[1, -1, 1]$.

$$\begin{aligned}\delta_2 &= e, \nabla_{W_2} L = e h^T = \begin{bmatrix} 1 & 0 \\ -1 & 0 \\ 1 & 0 \end{bmatrix} \\ \delta_1 &= \left(W_2^T e \right) \odot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix} \\ \nabla_{W_1} L &= \delta_1 x^T = \begin{bmatrix} 2 & 0 & -2 \\ 0 & 0 & 0 \end{bmatrix}\end{aligned}$$

The output-weight gradient is error $\times$ hidden activation transposed. The hidden error uses the output weights and then the ReLU derivative; a negative pre-activation blocks that component's gradient.

Bias gradients are the corresponding error vectors. Updating **both weights and biases** with learning rate 0.1 gives the next prediction **[0.26, 0.14, 0.26]**. The loss $\frac{1}{2} \|\text{prediction} - \text{target}\|^2$ falls from **1.5 to 0.4374**.

[PyTorch autograd](#) performs this chain-rule propagation automatically; the notebook checks the hand calculation.

5. Read the training loop as a calculation

Each batch computes predictions and a loss, clears previously accumulated parameter gradients, differentiates the current loss, and applies an optimizer update:

```
prediction = model(inputs)
loss = loss_function(prediction, targets)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

`loss.backward()` computes gradients; `optimizer.step()` changes parameters. Without clearing gradients, successive backward passes accumulate them.

Evaluate validation loss after an epoch and retain the weights at its minimum. Keep the test set out of parameter fitting and model selection. Compute prediction metrics after undoing output standardization.

In dynamic data, assign whole trajectories to train, validation, or test. A random split of neighboring transitions can make the evaluation less independent.

Original explanations: PyTorch [training loop](#) and [model construction](#).

6. Diagnose the held-out predictions

Suppose a reference row is $[0.6, 0.4, 0.2]$ mol/L and the predictor returns $[0.62, 0.39, 0.19]$ mol/L.

The component errors are $[0.02, -0.01, -0.01]$ mol/L. Both sums equal 1.2, so the concentration-sum residual is **zero**. Correct total concentration does not make each component exact.

Check: which quantities should a test report include, and which data may be used to choose the model?

Answer check: report A/B/C RMSE in physical units, concentration-sum residual, and minimum predicted concentration. Fit on train, choose with validation, and leave test out of both steps.

These are independent prediction diagnostics. Week 3 adds one-step and trajectory errors; Week 4 studies physical constraints. Selecting an operating point is a later task. Return to the [Week 2 core lab](#).